

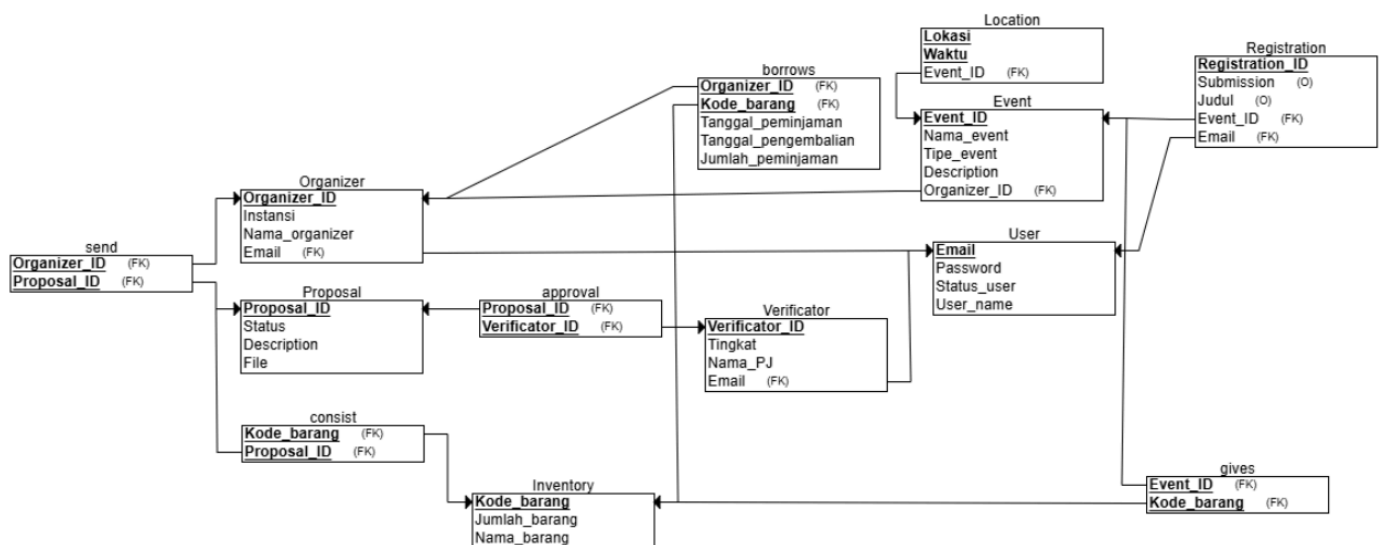
Eksplanasi Singkat: Campus Event Manager

Topik: Campus Event Management System

Kelompok: 11

Github: github.com/Dzaky-Ar/DBProject_Group11_CampusEventManager

1. Relational Schema dan Deskripsi Tabel



Dalam sistem ini, terdapat beberapa entitas yang saling berhubungan untuk mempermudah proses administrasi, seperti pengajuan proposal, registrasi peserta, pengelolaan inventaris, serta pengatur peminjaman fasilitas. Struktur dari *relational schema* terdiri dari tujuh tabel utama, yaitu Verificator, Proposal, Organizer, User, Registration, Event, Location, dan Inventaris, dan empat tabel relasi (yang dibuat dari relationship), yaitu consist, borrows, gives, dan send.

Setiap tabel memiliki peran spesifik dan saling berelasi untuk memastikan bahwa data-data dapat dikelola secara efisien, terstruktur, dan konsisten.

1. Verificator

Tabel Verificator digunakan untuk menyimpan data pihak penanggung jawab (PJ) atau verifikator dalam sistem. Informasi yang disimpan meliputi Verificator_ID, Nama_PJ, dan Email. Primary key dari tabel ini adalah

Verifier_ID. Tipe data yang digunakan berturut-turut adalah INT, VARCHAR(254), dan VARCHAR(254).

2. Proposal

Tabel Proposal digunakan untuk menyimpan data proposal yang diajukan oleh organizer. Informasi yang disimpan meliputi Proposal_ID, Status, Description, File, Organizer_ID, Verifier_ID, dan Kode_barang. Primary key dari tabel ini adalah Proposal_ID. Status disimpan dalam bentuk enum (*pending, approved, not approved*), sedangkan file proposal disimpan dalam bentuk PDF menggunakan tipe data BLOB. Tipe data yang digunakan secara berurutan adalah INT, ENUM, VARCHAR(254), BLOB, dan INT untuk atribut relasional.

3. Organizer

Tabel Organizer digunakan untuk menyimpan data panitia atau penyelenggara acara. Informasi yang disimpan meliputi Organizer_ID, Nama_organizer, Instansi, dan Email. Primary key dari tabel ini adalah Organizer_ID. Tipe data yang digunakan berturut-turut adalah INT, VARCHAR(100), VARCHAR(100), dan VARCHAR(254).

4. User

Tabel User digunakan untuk menyimpan data autentikasi dan informasi dasar seluruh pengguna sistem. Informasi yang disimpan meliputi Email, Password, Status_user, dan User_name. Primary key dari tabel ini adalah Email. Tipe data yang digunakan berturut-turut adalah VARCHAR(254), VARCHAR(100), ENUM, dan VARCHAR(254).

5. Registration

Tabel Registration digunakan untuk menyimpan data pendaftaran user terhadap suatu event. Informasi yang disimpan meliputi Registration_ID, Submission, Judul, Email, dan Event_ID. Primary key dari tabel ini adalah Registration_ID. Submission disimpan dalam bentuk file PDF menggunakan tipe data BLOB, sedangkan judul menggunakan VARCHAR(254). Atribut Email dan Event_ID berfungsi sebagai foreign key.

6. Event

Tabel Event digunakan untuk menyimpan informasi detail acara yang diselenggarakan. Informasi yang disimpan meliputi Event_ID, Nama_event, Tipe_event, Description, dan Organizer_ID. Primary key dari tabel ini adalah Event_ID. Tipe data yang digunakan berturut-turut adalah INT, VARCHAR(254), VARCHAR(50), VARCHAR(254), dan INT

7. Location

Tabel Location digunakan untuk menyimpan informasi lokasi dan waktu pelaksanaan acara. Informasi yang disimpan meliputi Lokasi, Waktu, dan Event_ID. Primary key dari tabel ini merupakan gabungan dari Lokasi dan Waktu. Tipe data yang digunakan berturut-turut adalah VARCHAR(200), DATE, dan INT.

8. Inventory (inventaris)

Tabel Inventory digunakan untuk menyimpan data barang yang tersedia. Informasi yang disimpan meliputi Kode_barang, Nama_barang, Jumlah_barang, dan Verificator_ID. Primary key dari tabel ini adalah Kode_barang. Tipe data yang digunakan berturut-turut adalah INT, VARCHAR(200), INT, dan INT

9. Borrows

Tabel Borrows digunakan untuk menyimpan data peminjaman barang dari inventaris. Informasi yang disimpan meliputi Tanggal_peminjaman, Tanggal_pengembalian, Jumlah_peminjaman, Organizer_ID, dan Kode_barang. Tabel ini tidak memiliki primary key tunggal dan menggunakan foreign key untuk menghubungkan data peminjaman dengan organizer dan inventaris. Tipe data yang digunakan berturut-turut adalah DATE, DATE, dan INT.

2. Implementasi SQL

```
-- Database
CREATE DATABASE IF NOT EXISTS cem;
USE cem;

-- =====
-- TABLE: user
-- =====
CREATE TABLE `user` (
  `Email` VARCHAR(254) NOT NULL,
  `Password` VARCHAR(100) NOT NULL,
  `Status_user` ENUM('1','2','3') DEFAULT NULL,
  `User_name` VARCHAR(254) NOT NULL,
  PRIMARY KEY (`Email`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;

-- =====
-- TABLE: verifactor
-- =====
CREATE TABLE `verifactor` (
  `Verifactor_ID` INT NOT NULL AUTO_INCREMENT,
  `Nama_PJ` VARCHAR(254) NOT NULL,
  `Email` VARCHAR(254) NOT NULL,
  PRIMARY KEY (`Verifactor_ID`),
  KEY `Email` (`Email`),
  CONSTRAINT `verifactor_ibfk_1`
  FOREIGN KEY (`Email`) REFERENCES `user` (`Email`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;

-- =====
-- TABLE: organizer
-- =====
CREATE TABLE `organizer` (
  `Organizer_ID` INT NOT NULL AUTO_INCREMENT,
  `Nama_organizer` VARCHAR(100) NOT NULL,
  `Instansi` VARCHAR(100) NOT NULL,
  `Email` VARCHAR(254) NOT NULL,
  PRIMARY KEY (`Organizer_ID`),
  KEY `Email` (`Email`),
  CONSTRAINT `organizer_ibfk_1`
  FOREIGN KEY (`Email`) REFERENCES `user` (`Email`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

```

-- =====
-- TABLE: event
-- =====
CREATE TABLE `event` (
  `Event_ID` INT NOT NULL AUTO_INCREMENT,
  `Nama_event` VARCHAR(254) NOT NULL,
  `Tipe_event` VARCHAR(50) DEFAULT NULL,
  `Description` VARCHAR(254) DEFAULT NULL,
  `Organizer_ID` INT DEFAULT NULL,
  PRIMARY KEY (`Event_ID`),
  KEY `Organizer_ID` (`Organizer_ID`),
  CONSTRAINT `event_ibfk_1`
    FOREIGN KEY (`Organizer_ID`) REFERENCES `organizer` (`Organizer_ID`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;

-- =====
-- TABLE: inventory
-- =====
CREATE TABLE `inventory` (
  `Kode_barang` INT NOT NULL AUTO_INCREMENT,
  `Nama_barang` VARCHAR(200) NOT NULL,
  `Jumlah_barang` INT NOT NULL,
  `Verifierator_ID` INT NOT NULL,
  PRIMARY KEY (`Kode_barang`),
  KEY `Verifierator_ID` (`Verifierator_ID`),
  CONSTRAINT `inventory_ibfk_1`
    FOREIGN KEY (`Verifierator_ID`) REFERENCES `verificator` (`Verifierator_ID`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;

-- =====
-- TABLE: proposal
-- =====
CREATE TABLE `proposal` (
  `Proposal_ID` INT NOT NULL AUTO_INCREMENT,
  `Status` ENUM('approved','pending','not approved') NOT NULL DEFAULT 'pending',
  `Description` VARCHAR(254) DEFAULT NULL,
  `File` BLOB DEFAULT NULL,
  `Organizer_ID` INT DEFAULT NULL,
  `Verifierator_ID` INT DEFAULT NULL,
  `Kode_barang` INT DEFAULT NULL,
  PRIMARY KEY (`Proposal_ID`),
  KEY `Organizer_ID` (`Organizer_ID`),
  KEY `Verifierator_ID` (`Verifierator_ID`),
  KEY `Kode_barang` (`Kode_barang`),
  CONSTRAINT `proposal_ibfk_1`
    FOREIGN KEY (`Organizer_ID`) REFERENCES `organizer` (`Organizer_ID`),
  CONSTRAINT `proposal_ibfk_2`
    FOREIGN KEY (`Verifierator_ID`) REFERENCES `verificator` (`Verifierator_ID`),
  CONSTRAINT `proposal_ibfk_3`

```

```
FOREIGN KEY (`Kode_barang`) REFERENCES `inventory` (`Kode_barang`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

```
-- =====  
-- TABLE: registration  
-- =====
```

```
CREATE TABLE `registration` (  
  `Registration_ID` INT NOT NULL AUTO_INCREMENT,  
  `Submission` BLOB DEFAULT NULL,  
  `Judul` VARCHAR(254) DEFAULT NULL,  
  `Email` VARCHAR(254) NOT NULL,  
  `Event_ID` INT NOT NULL,  
  PRIMARY KEY (`Registration_ID`),  
  KEY `Email` (`Email`),  
  KEY `Event_ID` (`Event_ID`),  
  CONSTRAINT `registration_ibfk_1`  
    FOREIGN KEY (`Email`) REFERENCES `user` (`Email`),  
  CONSTRAINT `registration_ibfk_2`  
    FOREIGN KEY (`Event_ID`) REFERENCES `event` (`Event_ID`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

```
-- =====  
-- TABLE: location  
-- =====
```

```
CREATE TABLE `location` (  
  `Lokasi` VARCHAR(200) NOT NULL,  
  `Waktu` DATE NOT NULL,  
  `Event_ID` INT NOT NULL,  
  PRIMARY KEY (`Lokasi`, `Waktu`),  
  KEY `Event_ID` (`Event_ID`),  
  CONSTRAINT `location_ibfk_1`  
    FOREIGN KEY (`Event_ID`) REFERENCES `event` (`Event_ID`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

```
-- =====  
-- TABLE: gives  
-- =====
```

```
CREATE TABLE `gives` (  
  `Kode_barang` INT NOT NULL,  
  `Event_ID` INT NOT NULL,  
  KEY `Kode_barang` (`Kode_barang`),  
  KEY `Event_ID` (`Event_ID`),  
  CONSTRAINT `gives_ibfk_1`  
    FOREIGN KEY (`Kode_barang`) REFERENCES `inventory` (`Kode_barang`),  
  CONSTRAINT `gives_ibfk_2`  
    FOREIGN KEY (`Event_ID`) REFERENCES `event` (`Event_ID`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

```

-- =====
-- TABLE: borrows
-- =====
CREATE TABLE `borrows` (
  `Tanggal_peminjaman` DATE NOT NULL,
  `Tanggal_pengembalian` DATE NOT NULL,
  `Jumlah_peminjaman` INT NOT NULL,
  `Organizer_ID` INT NOT NULL,
  `Kode_barang` INT NOT NULL,
  KEY `Organizer_ID` (`Organizer_ID`),
  KEY `Kode_barang` (`Kode_barang`),
  CONSTRAINT `borrows_ibfk_1`
    FOREIGN KEY (`Organizer_ID`) REFERENCES `organizer` (`Organizer_ID`),
  CONSTRAINT `borrows_ibfk_2`
    FOREIGN KEY (`Kode_barang`) REFERENCES `inventory` (`Kode_barang`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;

-- =====
-- TRIGGER: update inventory before borrow
-- =====
DELIMITER $$

CREATE TRIGGER `trg_update_inventory_before_insert`
BEFORE INSERT ON `borrows`
FOR EACH ROW
BEGIN
  IF (
    SELECT Jumlah_barang
    FROM inventory
    WHERE Kode_barang = NEW.Kode_barang
  ) < NEW.Jumlah_peminjaman THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Not enough stock in inventory.';
  END IF;

  UPDATE inventory
  SET Jumlah_barang = Jumlah_barang - NEW.Jumlah_peminjaman
  WHERE Kode_barang = NEW.Kode_barang;
END$$

DELIMITER ;

```

